The beginning

Publishing gave the questions a place to begin.

I founded and operated kailor insider as an independent publishing initiative in 2024. I wanted a place to research artificial intelligence and emerging technology, work through what developments meant, and communicate ideas in a useful, considered way. The work involved editorial research, writing, digital content development and the practical responsibility of running a publication.

That foundation has grown beyond AI coverage. Technology, software engineering, applied innovation and the relationship between technology and business all belong in its editorial scope. Building the publication also meant building the systems around it: how writing moves through review, how readers discover ideas, and how participation can be supported responsibly.

The evolution

From understanding technology to applying it.

Researching and explaining technology kept bringing me back to practical questions: what does a business actually need, which constraints matter, and what is worth building? That experience contributed to the creation of kailor in 2025. I developed it into a founder-led technology consultancy and engineering company, carrying the same curiosity into operational problems and delivery.

The publication continues independently. kailor insider communicates, publishes and connects ideas; kailor evaluates, engineers and delivers technology. I founded and operate these ventures as ongoing businesses, with software platforms supporting their different responsibilities.

kailor today

Understand the problem. Engineer an appropriate response.

My consulting work starts with people, business needs and existing processes. I examine operational challenges, evaluate opportunities and recommend an approach before deciding what to implement. That can mean a process improvement, an established tool, an integration, conventional software, automation or a custom AI system. AI is an option within the work, rather than a condition for taking it on.

I bring together four disciplines: technology consultancy; software engineering across applications, websites, backend services, APIs and databases; practical AI and automation using language models, retrieval and supervised workflows where appropriate; and cloud infrastructure, deployment systems, integrations and operational reliability. Discovery, design, building and continuing support belong to the same delivery practice.

The kailor platform

A project keeps its context from discovery to decision.

The consulting application is one part of kailor. Its React and TypeScript interface gives each owned project an independent workspace for Chat, Discovery, Canvas, Analysis and Sources. Discovery answers and conversations inform attributed evidence and requirements. Confirmed facts, assumptions, proposed corrections and unknowns remain distinguishable, so an attractive recommendation cannot become evidence for itself.

The Express backend uses shared TypeScript and Zod contracts, Prisma and PostgreSQL. Mutations check ownership and revisions inside transactions; conflicting edits return a conflict rather than silently overwriting context. Current solution canvases can evolve while saved reports remain immutable snapshots. Optional, budgeted Cloudflare inference proposes structured output through validation and confirmation boundaries; it cannot directly write confirmed business facts.

The current local implementation supports comparing and selecting one or several solutions, reviewing their combined scope and submitting an immutable consultancy request. A founder inbox receives those requests for human review. A request is an intake for discussion, not a commissioned contract. Scoping, binding proposals, commissioning, billing and implementation tracking remain developing commercial workflows; engineering and client delivery are human-led consultancy work.

The public company website at kailor.co.za is live. Its static Pages build excludes the private application, backend, database and AI runtime. The upcoming app launch is a separate milestone. Source inspection here establishes implementation, not a newly verified public consulting application or client outcomes.

kailor insider today

A publication built around writing, reading and participation.

I am developing kailor insider as a complete publication and social reading experience. Its Studio supports rich Tiptap authoring, autosave, private drafts, previews, revisions, review, scheduling and publication controls. Editorial research and publishing operations sit alongside the engineering of discovery feeds, topics, full-text search and explainable recommendations.

Implemented account and participation models cover profiles, preferences, follows, saves and personal lists, reactions, threaded discussion, notifications, moderation and one-to-one messages. These are current application capabilities, not evidence of an open public community or audience scale. The application is currently owner-only, with registration closed. Scoped recipient grants permit limited article reading without opening the wider application.

The current architecture replaces the earlier separate Express publishing API with React Router server rendering inside an independent Cloudflare Worker. TypeScript services use PostgreSQL and forward SQL migrations; media is private through an R2 adapter or the implemented PostgreSQL storage alternative. This has its own application lifecycle, authentication, database and release procedure, separate from kailor.

Source repositories (authorised access may be required)

kailor: https://github.com/keenosmith-del/kailor

kailor insider: https://github.com/keenosmith-del/kailor-insider

Evidence: local source review, 26 October 2026. kailor revision 3d6d459; kailor insider revision f5f3c2f. Architecture, article/auth/media/database services, canvas implementation and deployment reports inspected. Company suites were not rerun for this portfolio iteration.

Engineering both systems

Durable records and explicit boundaries.

Both systems use relational persistence, but their domains differ. kailor preserves evidence provenance, project ownership, revision-aware recommendations and historical report snapshots. kailor insider locks article writes, compares versions and updates content, revisions, topics and search atomically. Its database wrapper uses savepoints for nested transactions, avoiding partial changes when an inner operation fails.

Authentication is enforced independently. kailor uses hashed, revocable session tokens, account and guest ownership checks, origin validation and separate guest authority. kailor insider combines owner checks with hashed sessions, password verification, TOTP and recovery controls. Rich publication documents are allowlisted and escaped; image signatures, dimensions and ownership are checked before private storage or access. Media and database changes still need reconciliation when separate stores cannot share a transaction.

Reliability work includes bounded database pools and pagination, revision conflicts, backup and restore tooling, and explicit deployment checks. Optional AI writing in kailor insider is prepared as a bounded job and requires owner review and approval before publication. Internal automated identities and development simulations are labelled; their activity is not evidence of a real readership.

I reviewed current source and repository verification records for this case study; I did not rerun the company applications' suites. kailor insider has local verification and an isolated staging deployment, but its latest staging report records password-sign-in CPU failures and SSR/autosave runtime limits. It is not production-ready. Hosted AI, SMTP and OAuth operation are not established here. I have omitted obsolete test counts rather than presenting old results as current.

What connects them

A shared history. Distinct responsibilities.

Publishing taught me to investigate, explain and question assumptions. Consultancy asks me to apply that discipline when evaluating and engineering practical systems. Today, kailor insider remains a dedicated editorial business and kailor remains a broader technology consultancy. Neither is a small feature inside the other.

They share an identity, an interest in useful technology and a considered design language. They retain independent authentication, data stores, deployment boundaries and publishing or consultancy workflows. The connection is in the thinking and history; technical integration is not assumed.